INDEX

1. ENERGY METERS

- Under energy meters we are considering
    - Three-Phase
    - Single-Phase
    - Virtual Switch
    - Smart Plug
- Relay will be turned ON and OFF by sending the MQTT request.
- This version will be sent after the device checks for the update and extracts the version from the link provided.
- The device will send the data every 5 seconds.
- For all the devices the boot data will be sent to the topic
    - **three-phase/command/{deviceId}**
- Followed by the device restart status
    - **{"restartReason":0,"restartCount":6}**

Smart - Plug:

★ For the smart plug is we press and hold the button for 10 seconds wifi config will be reset

★ Factory reset can only happen through mqtt.

★ In the smart plug ,when the device factory resets all the values will be erased except Energy and DeviceId.

★ If there is no app then the device is directed to a captive portal to operate the device functionality. In the case of the presence of the app then we will be redirected to the app.

★ It stays for 10 min in the captive portal.after that we need to reconnect to the plug. The same functionality is applicable even if there is no wifi connection.

★ Here we can set the device to configuration mode to use the captive portal.

★ If the LED is blinking slow then there is no connectivity. If there is a rapid blinking with the leds it indicates that the device is in AP mode.

★ And the device stays in AP mode for 3 min.

★ In case of no wifi the plug tries for the wifi connection only if we set the plug to configuration we can change the wifi or operate it in captive portal.

1.1 WI-FI Configuration:

- ➢ Press the reset button for 10 seconds so that we will be able to configure the wifi details.

1.2 MQTT Message Format and Respective Topics:

1.2.1 Three phase and Single phase:

- The message format consisting of the device details is being sent to the below MQTT topic.
  MQTT_TOPIC_ENERGY **"energy-meter/three-phase"**
- The relay changes- such as relay on and off are being sent through this Mqtt topic.
  MQTT_TOPIC_RELAY "**energy-meter/three-phase/command/deviceId**"
- To get the relay status back we need to use **"energy-meter/three-phase/status**
- To send the relay on off message to the device we need to use:
  - **Topic: energy-meter/three-phase/command/{deviceid} -** for three phase
  - **{"entityId":"{devcieId}","relayStatus":"ON"}**
- Using the same above topic we can see the device boot data.

❖ To send the commands to the device we need to subscribe to the following topic

- ➢ **single-phase/{deviceId}/config/command**
- ➢ **three-phase/{deviceId}/config/command**

Offline timer :

- ➢ **TOPIC: three-phase/{device_id}/timer/command**
- ➢ **Payload: {"timer":"1", "action":"10"}**
- ➢ **Timer : time in minutes**
- ➢ **Action: 10 - relay position 1, 0 - off**
  - **11 - relay position 1, 1 - on**
  - **0 - disable offline timer**
- ➢ **To Disable the timer send the below message to the same topic**
  - **{"timer":"0","action":"0"}**

❖ In the topic **three-phase/{deviceid}/ping/status**

- ➢ We can see the below message
- ➢ **{"switch":[1],"cause":"ButtonAction","timer":{"remaining":30,"action":10}}**
  - Where **"switch":[1]** indicate plug is **ON**
  - And **"switch":[0] i**ndicates **OFF** condition

- **"cause":"ButtonAction" -** indicates the cause of the action that has occurred previously.
- **"Timer":{"remaining":30,"action":10}**
- **Here** remaining is the time in **Seconds**
- Action indicates the relay action as in **10 -** relay position **1** and **status off(0)**

➢

1.2.2 Smart Plug and Virtual Switch:

- Some functionalities vary for the smart plugs.
- As to send the relay status both the virtual switch and smart plug use the same topic as given below.
  - **Topic: smart-switch/command/{deviceid}**
  - **{"entityId":"BSP10000019","relayStatus":"ON"}**
- And the data is published to
  - **Topic: smart-switch/data**
- For boot data the topic is:
  - **smart-switch/{deviceid}/boot/status**
- For offline timer:
  - **smart-switch/{deviceid}/timer/command**

Offline timer :

TOPIC: smart-switch/{device_id}/timer/command

Payload: {"timer":"1", "action":"10"}

- **Timer : time in minutes**
- **Action: 10 - relay position 1, 0 - off**
  - **11 - relay position 1, 1 - on**
  - **0 - disable offline timer**
- **To Disable the timer send the below message to the same topic**
  - **{"timer":"0","action":"0"}**

2.TUNABLE LIGHTS

2.1 WI-Fi Configuration:

- To restart the device we need to run a power cycle for 10 times.i.e, we need to completely turn OFF the device and then we need to turn ON, this same process needs to be done for 10 times.(the number of times will be changed in the near future)
- The same method is used to configure the wifi as well.

2.2 Operation:

- We have implemented tunable lights. These lights have been operated by the device when needed to increase and decrease the brightness of the light.
- The light intensity is ranged from **0 - 100**. 0- being the lowest(off state) and 100- being the Highest.
- These lights can be controlled via Mqtt or via app as well.

2.3 MQTT Messages and Respective Topics:

- The topic of the tunable lights is as follows:
    - **tunable-light/{deviceid}/light/command**
    - The message format to send the brightness is as follows
    - **{"entityId":"deviceId","type": "brightness","value": 0}**
    - To get the status back we need to use
        - **“tunable-light/{deviceid}/light/command”.**
    - To get the status we need to check at
        - **“tunable-light/{deviceid}/light/status”.**
- To send the commands to the device we need to subscribe to the following topic
    - **tunable-light/{deviceId}/config/command**
- For **RGBW** we need to use the following command to change the status of the lights
    - **rgbw-light/{deviceid}/light/command**
    - **{"type": "colour","colour": [0, 0, 255, 0]}**

        - **The indices are followed in color as**
        - **1st one for RED**
        - **2nd one for GREEN**
        - **3rd one for BLUE**
        - **4th one for WHITE**
    - For brightness we can follow the same command.
    - **Just remember to change the topic to “rgbw-light” from tunable-light**

**** The point to note is that the values are given in decimal form(8 bit) 0 - 255.(255 being highest)****

- For this topic if we send the following message we will receive all the information regarding the device such as device Id,DeviceHostname,Ip address etc;
    - **{“action”:”reportinfo”}**
- To get the setting that we have been using then we have to follow the following command.
    - **{“action”:”reportsettings”}**
- To check for ota updates we need to request using the following command
    - **{“action”:”checkforupdate”}**
- Reset and restart can also be continued with the same command.
- Reset energy: this function will set all the energy values to zero.
    - **{“action”:”resetenergy”}**
- We can disconnect from the current wifi and set them again
    - **{“action”:”disconnect”}- disconnect from wifi**
- Reconnect to the same Wifi
    - **{“action”:”reconnect”}**
- Reset the device completely. It sets all the values for the device to zero.
    - **{“action”:”factoryset”}**
- Restart the device.
    - **{“action”:”restart”}**
- **Using config we can change the device parameters that are present in report settings.**
- **The command to be sent is as follows:**
    - **{“config”:{“eota”:”1”}}**
- **In the place of eota we can change any of the parameters in the report settings functionality using this config key type.**
- **Eota:**

- For updating the bin file we have been using the eota method. This method includes an ota update method that is just used for the update while developing**"not to be used in field" -** as for the field we will be using the existing method.
- This needs to be enabled before activation for that we need to send the following command.
    - **{"action":"enable_eota"}**
- To disable we follow
    - **{"action":"disable_eota"}**
- We need to make sure that we are disabling this as soon as updated.
- To dump the code we need to check the ip address(it will be given in the report info settings) and search in a new tab for **"ip address/updates".**
- To verify it is off after disable make sure that the page shows no updates available.
- **"If the eota is enabled then we can't send the ota in real time"**

3. SWITCHES

3.1 TOUCH PANNELS

3.1.1 Wifi Configuration:

- For configuring the wifi for touch panels we need to press and hold any button on the panel for 15 to 20 sec approx. till all the leds of the switches will blink and then we need to press on that button again(same button) for the device to enter into config mode.
- A light press(2 sec) will initiate a blink routine. That is for remote control configuration.**"we are currently not using it no need to worry about that"**

3.1.2 User modifiable functions:

- User can change and modify the values of the following.
    - Switches ON and OFF
    - Led colors - that are represented for the Switches(for both ON and OFF individually)
    - Brightness time for the switches.i.e after how much time brightness of the device can be reduced.
    - Brightness of the switches itself.the value of the brightness range from 0 - 255, 255 - being the highest.

3.1.3 MQTT topics:

- To send the MQTT command we use the below topic and message format.
    - **touch-panel/{deviceID}/switch/command**
    - **{"entityId":"BS400000001","type":"switch","value":"50"}**

      Here we need to check for the below details:
        - For the key ‘value’ we will be using 2 indices as given in the above example 5 and 0.
        - Here 5 is the switch index. The index value will range on the number of switches present.
        - And the second index is for switch state ON(1) or OFF(0)
        - To ON 4th switch we give the parameter as “value”:”41”
        - To off the 10th switch its “value”:”100”
        - In the case of Fan, Fan is the 5th switch and we will be using a dimmer to control its speed.

- for switch status “**touch-panel/{deviceID}/switch/status”**

- to send the dimmer update to the fan we need to follow the following topic and message format

- **touch-panel/{deviceID}/switch/command**
- **{"entityId":"BS400000001","type":"dimmer","dimmer":"5","value":"5"}**
- The value field varies from 1 - 5 steps

- To change the backlight of the device we need to
    - **touch-panel/BS400000001/backlight/command**
    - **{"bklt": [102, 204, 0, 0, 102, 255, 100, 10, 100]}**

- here the first three indexes are for the color of the panel when the device is in ON condition,followed by the OFF condition.
- The 7th index is for the ON State brightness of the switches and 9th is the same for OFF state.
- 8th index is for the time interval to adjust the brightness of switches in the touch panel.
- For status we need to check “**touch-panel/BS400000001/backlight/status”**
    - **three-phase/{deviceId}/config/command**

3.2 RETRO FIT

3.2.1 MQTT topics:

- To send the MQTT command we use the below topic and message format.
    - **node-switch/{deviceID}/switch/command**
    - **{"entityId":"BS400000001","type":"switch","value":"50"}**

      Here we need to check for the below details:
        - For the key ‘value’ we will be using 2 indices as given in the above example 5 and 0.
        - Here 5 is the switch index. The index value will range on the number of switches present.
        - And the second index is for switch state ON(1) or OFF(0)
        - To ON 4th switch we give the parameter as “value”:”41”
        - To off the 10th switch its “value”:”100”
        - In the case of Fan, Fan is the 5th switch and we will be using a dimmer to control its speed.

- for switch status “**node-switch/{deviceID}/switch/status”**

- to send the dimmer update to the fan we need to follow the following topic and message format

- **node-switch/{deviceID}/switch/command**
- **{"entityId":"BS400000001","type":"dimmer","dimmer":"5","value":"5"}**
- The value field varies from 1 - 5 steps

-For status we need to check “**node-switch/BS400000001/backlight/status”**

- **node-switch/{deviceId}/config/command**

3.3 MODULAR TOUCH:

3.3.1 MQTT topics:

- To send the MQTT command we use the below topic and message format.
    - **modular-touch/{deviceID}/switch/command**
    - **{"entityId":"BS400000001","type":"switch","value":"50"}**

      Here we need to check for the below details:
        - For the key ‘value’ we will be using 2 indices as given in the above example 5 and 0.
        - Here 5 is the switch index. The index value will range on the number of switches present.
        - And the second index is for switch state ON(1) or OFF(0)
        - To ON 4th switch we give the parameter as “value”:”41”
        - To off the 10th switch its “value”:”100”

- In the case of Fan, Fan is the 5th switch and we will be using a dimmer to control its speed.

- for switch status “**modular-touch/{deviceID}/switch/status”**

- to send the dimmer update to the fan we need to follow the following topic and message format

- **modular-touch/{deviceID}/switch/command**
- **{"entityId":"BS400000001","type":"dimmer","dimmer":"5","value":"5"}**
- The value field varies from 1 - 5 steps

-For status we need to check “**node-switch/BS400000001/backlight/status”**

- **modular-touch/{deviceId}/config/command**

4. RS485 WORKFLOW

MQTT topic : rs485/{deviceID}/workflow/config
Message Format :[{"id":2,"data":"current_y,3;240,10;B;R,1,0"}]

"**id**": work flow number you want to add
if id is 2 here, then this indicates that we are creating work flow 2.

"**data**": has whole around 8 parameters
1st - parameter which we want to monitor
2nd - parameter value we want to monitor
3rd - currently leave it as it is
4th - time period we want to check in seconds.
5th - we can to check the mentioned value Below(B), Above(A) or Equal(E) to the value that we have mentioned.
6th - Relay or dimmer which ever we want to toggle(R - relay, D- Dimmer) - currently using only relay
7th - index - index of the relay
8th - value of the relay
0 - OFF
1 - ON

ACTION AND CONFIGURATION COMMANDS:

- For configuration data the topic is
  - **smart-switch/{deviceid}/config/status**
- For this topic if we send the following message we will receive all the information regarding the device such as device Id,DeviceHostname,Ip address etc;
  - **{“action”:”reportinfo”}**
- To get the setting that we have been using then we have to follow the following command.
  - **{“action”:”reportsettings”}**
- To check for ota updates we need to request using the following command
  - **{“action”:”checkforupdate”}**
- Reset and restart can also be continued with the same command.
- Reset energy: this function will set all the energy values to zero.
  - **{“action”:”resetenergy”}**
- We can disconnect from the current wifi and set them again
  - **{“action”:”disconnect”}- disconnect from wifi**
- Reconnect to the same Wifi
  - **{“action”:”reconnect”}**
- Reset the device completely. It sets all the values for the device to zero.
  - **{“action”:”factoryset”}**
- Restart the device.
  - **{“action”:”restart”}**
- Using config we can change the device parameters that are present in report settings.
- The command to be sent is as follows:
  - **{“config”:{“eota”:”1”}}**
- In the place of eota we can change any of the parameters in the report settings functionality using this config key type.

EOTA

- For updating the bin file we have been using the eota method. This method includes an ota update method that is just used for the update while developing**"not to be used in field" -** as for the field we will be using the existing method.
- This needs to be enabled before activation for that we need to send the following command.
  - **{"action":"enable_eota"}**
- To disable we follow
  - **{"action":"disable_eota"}**
- We need to make sure that we are disabling this as soon as updated.
- To dump the code we need to check the ip address(it will be given in the report info settings) and search in a new tab for **"ip address/updates".**
- To verify it is off after disable make sure that the page shows no updates available.
- **"If the eota is enabled then we can't send the ota in real time"**

- **We are sending the restart conditions when the device is restarted to know what are the reasons to get to know why the device is restarted.**
- **This message will be a follow up message for the boot data.**
- **boot data contains restart count as well.**
- **{device-type}/{deviceid}/boot/status**
- **The message format is as stated.**
    - **{"restartReason":9,"restartCount":1}**

CUSTOM_RESET_HARDWARE 1
CUSTOM_RESET_WEB 2
CUSTOM_RESET_TERMINAL 3
CUSTOM_RESET_MQTT 4
CUSTOM_RESET_RPC 5
CUSTOM_RESET_OTA 6
CUSTOM_RESET_UPGRADE 7
CUSTOM_RESET_FACTORY 8
CUSTOM_RESET_WIFI_CONFIG 9

- These will be sent with the boot data and restart count for the devices will also be sent as how many times the device is being restarted.
- “All this information will go back to default on factory reset.”

➔ Hardware (1) : if there is any button action from Hardware
➔ WEB (2) : if the reset occurred due to a command from the web dashboard.
➔ Terminal(3) : using serial communication if reset has taken place (mostly occured while debugging).
➔ MQTT(4) : if a command has been sent through mqtt for restart/reset of the device.
➔ RPC:(5): ignore currently not in use
➔ OTA(6): whenever we give an OTA or EOTA update the device restarts.
➔ Upgrade(7) : ignore
➔ Factory(8): complete device reset
➔ Wifi(9): whenever we set wifi credentials device restarts.